

Lesson 8: Async Programming and Promises

Intro to Programming — Code the Dream

Session Overview

By the end of this session, you will be able to:

- Explain the difference between synchronous and asynchronous code
- Describe what a Promise is and its three possible states
- Prevent a form's default page-refresh behavior with `preventDefault`
- Read form values from a submit event
- Add and remove items from the DOM dynamically

Explore

Synchronous Code

Synchronous code runs **one step at a time**, in order.

```
console.log("Step 1");  
console.log("Step 2");  
console.log("Step 3");  
// Always prints: Step 1, Step 2, Step 3
```

Each line **waits** for the line above it to finish.

Real-life example: a phone call — you wait for the other person to finish before you speak.

Asynchronous Code

Asynchronous code lets you **start a task and keep moving** — you don't wait.

```
console.log("Order placed");

setTimeout(function () {
  console.log("Food is ready!");
}, 2000);

console.log("Chatting with friends...");
```

Output: Order placed → Chatting with friends... → (2 sec) → Food is ready!

Real-life example: a text message — you send it and do other things while you wait for a reply.

Why Async Matters for the Web

Web pages often need to wait for things:

- Loading data from a server
- Waiting for a user to submit a form
- Fetching an image or a file

If those tasks were **synchronous**, the entire page would **freeze** while waiting.

Async code keeps the page **interactive** — users can still click and scroll.

Check for Understanding

You visit a website and click a button to load new data. The page stays usable while the data loads. Which type of code is handling that?

- A) Synchronous — it runs one step at a time
- B) Asynchronous — it starts the task and keeps the page responsive
- C) Neither — JavaScript cannot load data from servers
- D) Both — synchronous and asynchronous always work together

What Is a Promise?

A **Promise** is an object that represents a result that **isn't ready yet**.

It has three possible states:

State	Meaning
Pending	Still waiting — no result yet
Fulfilled	Success — here is the value
Rejected	Something went wrong

Think of it like an online order confirmation: you don't have the item yet, but you have a *promise* it's coming.

Promises vs. Callbacks

We used **callbacks** in earlier lessons — functions passed to other functions.

```
// Callback style
setTimeout(function () {
  console.log("Done!");
}, 1000);
```

Promises give us a **cleaner way** to handle async results, especially when multiple async steps chain together.

"Callback hell" — deeply nested callbacks — is hard to read and debug. Promises help avoid that.

Check for Understanding

A Promise is in the "Pending" state. What does that mean?

- A) The operation failed and returned an error
- B) The operation succeeded and returned a value
- C) The operation has started but has not finished yet
- D) The Promise was never created

Forms and Event Listeners (Review)

We used event listeners in Lesson 7. This week we attach one to a **form**.

```
let btn = document.querySelector("button");  
btn.addEventListener("click", function () {  
  console.log("Clicked!");  
});
```

Forms fire a `"submit"` event when the user clicks the submit button.

We listen for it the same way — just with a different event name.

The Problem: Default Form Behavior

When a form submits, the browser **refreshes the page** by default.

That wipes out everything — the form values, the DOM changes, all of it.

We need to **stop that** before we do anything else.

```
form.addEventListener("submit", function (event) {  
    event.preventDefault(); // Stop the page refresh  
    // Now we can safely read the form values  
});
```

Always put `event.preventDefault()` as the **first line** inside the callback.

Check for Understanding

You add a submit event listener to a form, but the page refreshes every time you submit. What is most likely missing?

- A) The form does not have a `name` attribute
- B) `event.preventDefault()` was not called
- C) The event listener is using `"click"` instead of `"submit"`
- D) You cannot attach event listeners to forms

Apply

Core Activity: Build a Message Form

We'll build the message form from the assignment together.

Open your files:

- `index.html` — we'll add the form markup
- `index.js` — we'll handle the submit event

Tell me what to type!

Step 1 — Add the Form HTML

In `index.html`, above `<footer>`, add a new section:

```
<section>
  <h2>Leave a Message</h2>
  <form name="leave_message">
    <label>Name:
      <input type="text" name="userName" required>
    </label>
    <label>Email:
      <input type="email" name="usersEmail" required>
    </label>
    <label>Message:
      <textarea name="usersMessage" required></textarea>
    </label>
    <button type="submit">Submit</button>
  </form>
</section>
```


Step 2 — Handle the Submit Event

In `index.js`, select the form and listen for `"submit"`:

```
let messageForm = document.forms["leave_message"];

messageForm.addEventListener("submit", function (event) {
  event.preventDefault();

  let name = event.target.userName.value;
  let email = event.target.usersEmail.value;
  let message = event.target.usersMessage.value;

  console.log(name, email, message);
  messageForm.reset();
});
```

Save and try it. Check the browser console — you should see the values printed.

Step 3 — Display the Message on the Page

Inside the event listener, before `messageForm.reset()`:

```
let messageSection = document.querySelector("#messages");
let messageList = messageSection.querySelector("ul");

let newMessage = document.createElement("li");
newMessage.innerHTML =
  `<a href="mailto:${email}">${name}</a>
  <span>${message}</span>`;

messageList.appendChild(newMessage);
```

Ask the group: What does `mailto:` do when someone clicks the link?

Step 4 — Add a Remove Button

Still inside the event listener, before `messageList.appendChild(newMessage)` :

```
let removeButton = document.createElement("button");
removeButton.textContent = "remove";
removeButton.type = "button";

removeButton.addEventListener("click", function () {
  let entry = removeButton.parentNode;
  entry.remove();
});

newMessage.appendChild(removeButton);
```

Ask the group: What does `parentNode` refer to here?

Extension Activity

Only if students are ready and time allows.

Extension — Hide the Messages Section When Empty

Challenge: Hide the `#messages` section (including the heading) when there are no messages. Show it again when a message is added.

Think about it:

- How do you check whether the `<ul>` is empty?
- What CSS property would you toggle?
- Which events would trigger the check?

Hint: `messageList.children.length` tells you how many items are in the list.

Assignment Preview

This week you'll add a full message form to your portfolio:

- **HTML:** form with name, email, and message fields + a messages list
- **JS:** submit event listener with `preventDefault`
- **JS:** display each message as a list item with a `mailto:` link and a remove button
- **CSS:** style the form fields, labels, and buttons (including mobile sizing)
- **Nav:** add a link to the "Leave a Message" section in your `<nav>`

Stretch goals: hide the messages section when empty; add an edit button per message.

Tips for the Assignment

1. **Do the HTML first** — get the form visible in the browser before writing any JS
2. **Test `preventDefault` early** — submit once without it so you see the refresh, then add it
3. **Use `console.log` often** — log `name`, `email`, and `message` before building the DOM
4. **Build the remove button last** — get the message displaying first, then add the button
5. **Stuck on CSS?** Open DevTools and experiment directly on the element before writing code

Ask for hints, not answers: "My form resets but the list is empty — can you give me 3 hints?"

Wrap-Up

Zoom chat: Rate your confidence from 1 to 5

How comfortable do you feel with form events and updating the DOM?

Before next session:

- Work through the **lesson materials** (Odin Project — Asynchronous Code)
- Build the **message form** for your portfolio
- **Style** the form fields and buttons
- Post questions in the **Slack** `#discussion` channel anytime

Resources

Resource	What it's for
Odin Project — Asynchronous Code	Main reading for this lesson
MDN — <code>event.preventDefault</code>	Reference for stopping default behavior
MDN — <code>createElement</code> / <code>appendChild</code>	DOM manipulation reference
MDN — HTML Forms	Form elements and attributes
Slack <code>#discussion</code> channel	Questions between sessions

Next week: Fetch API — pulling real data from the internet using Promises!

Great work today!

Your portfolio now listens to users.

Questions? Reach out on Slack anytime.